

내가 고친 코드, AI가 설명해 줍니다

B3-2 추가 과제 · AI 코드 설명 도우미 만들기

과제 내용 → 실행 방법 → 결과

한 장으로 보는 과제



무엇을

코드를 고친 뒤 '무엇을 왜 바꿨는지' 쓰는 설명문을 AI가 대신 써 주는 터미널 프로그램 만들기



어떻게

Git에서 바뀐 내용을 모으고, 민감정보를 가린 뒤, Claude AI에 보내 정해진 형식의 설명문을 받기



결과

실제 AI로 설명문 생성 성공, 자동 테스트 27개 통과, 보너스 과제 3개 모두 완료

발표 순서 1 과제 내용 → 2 실행 방법 → 3 결과

1 과제는 무엇이었나

왜 필요한지, 꼭 알아야 할 개념, 요구사항

왜 이런 도구가 필요할까



지금까지

- 코드를 고친 뒤 리뷰어에게 보낼 설명을 직접 씁니다
- 바뀐 파일이 많으면 빠뜨리기 쉽습니다
- 사람마다 형식이 달라 읽기 어렵습니다



이 도구를 쓰면

- 명령 한 줄로 설명 초안이 나옵니다
- 바뀐 내용(diff)을 근거로 쓰니 빠뜨림이 줄어듭니다
- 항상 같은 4칸 형식이라 읽기 쉽습니다



핵심 메시지 AI에게 무엇을 보여 주느냐가 결과를 결정합니다. 그래서 바뀐 내용(diff), 수정 의도, 원하는 형식을 함께 보냅니다.

먼저 알아야 할 개념 4가지



Git · git status · git diff

Git은 코드 변경 기록 도구입니다. status는 '어떤 파일이 바뀌었나', diff는 '어떤 줄이 어떻게 바뀌었나'를 보여 줍니다.



AI API

프로그램이 인터넷으로 AI에게 질문을 보내고 답을 받는 창구입니다. 이 과제는 Anthropic의 Claude를 씁니다.



API Key와 환경변수

API Key는 AI 창구의 출입증입니다. 코드에 적으면 유출되므로 컴퓨터의 '환경변수'에만 넣어 둡니다.



프롬프트

AI에게 보내는 질문지입니다. 바뀐 내용, 수정 의도, 원하는 출력 형식을 함께 적어 보냅니다.

과제가 요구한 것



Git 변경 수집

status·diff 수집, 변경 없으면 안내 후 종료



수정 의도 입력

옵션(--intent) 또는 대화식 입력



AI API 연동

Key는 환경변수, 실패 원인 출력, 파라미터 옵션



설명문 생성

제목 1줄 + 4개 섹션 템플릿



형식 검증

제목 72자 이하, 섹션마다 불릿 1개 이상



안전 모드

민감정보 가리기, 전송량 제한

제출물: GitHub 저장소 + 사용 가이드(README.md) · 보너스: 실제 저장소 적용, 팀 컨벤션, 안전 모드 고도화

AI가 써야 하는 설명문의 모양

```
--- Description Title ---
feat: 사용자 세션 처리 보완

--- Description Body ---
## Summary
- 로그인 흐름이 안정적으로 동작합니다.
## What Changed
- session.py에 예외 처리를 추가했습니다.
## Why It Was Needed
- 빈 입력에도 세션이 만들어졌습니다.
## How to Verify
- 빈 문자열로 로그인해 오류를 확인합니다.
```

제목 1줄

50자 이내 권장, 최대 72자

Summary

한 줄 요약

What Changed

무엇이 바뀌었나 (파일 1~3개 언급)

Why It Was Needed

왜 바꿔야 했나

How to Verify

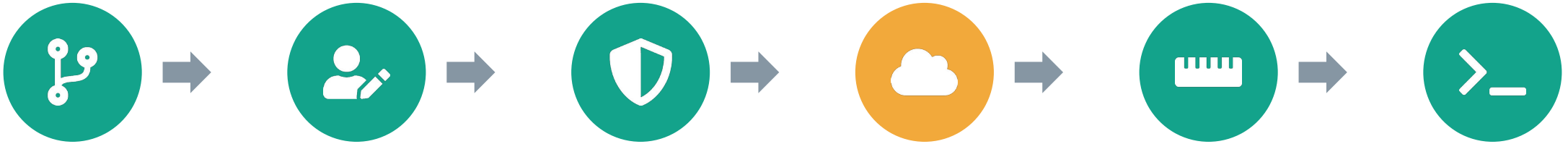
리뷰어가 어떻게 확인하나

2

어떻게 만들고 실행했나

전체 흐름, 프로그램 구조, 실행 방법, 안전장치

명령 한 줄 뒤에서 일어나는 6단계



① 수집

git status
git diff

② 의도 입력

무엇을 왜
고쳤는지

③ 안전 모드

비밀값 가리기
양 줄이기

④ AI 호출

Claude에게
질문 보내기

⑤ 검증

형식 검사
필요 시 재요청

⑥ 출력

터미널에
설명문 표시

```
$ python3 b3-2_추가/main.py describe --safe-mode --intent "로그인 예외 처리를 보완했다"
[INFO] Git status 수집 완료: 3개 파일 변경 감지   ·   [INFO] AI API 요청 중...   ·   [DONE] 코드 설명 생성 완료
```

프로그램은 역할별로 6개 파일로 나눴습니다

`cli.py` 지휘자

6단계 순서를 조립하고 옵션을 받습니다

`git_context.py` 수집 담당

git status·diff를 실행해 변경 내용을 모읍니다

`safe_mode.py` 보안 담당

비밀값을 가리고 보낼 양을 줄입니다

`ai_client.py` 통신 담당

Claude API를 호출하고 오류를 해석합니다

`generator.py` 편집 담당

질문지를 만들고 답을 검사·정리합니다

`convention.py` 규칙 담당

팀별 작성 규칙 파일을 읽습니다

직접 실행하는 방법: 3단계

1

설치

AI 라이브러리를 설치합니다

```
$ python3 -m pip install -r requirements.txt
```

2

API Key 설정

출입증을 환경변수에 넣습니다 (화면에 안 보이게)

```
$ read -s AI_API_KEY && export AI_API_KEY
```

3

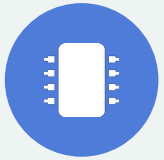
실행

Git 저장소 맨 위 폴더에서 실행합니다

```
$ python3 b3-2_추가/main.py describe --safe-mode
```

API 없이 연습하기 --mock: 가짜 응답으로 흐름만 확인 · --dry-run: AI에 보낼 질문지만 미리 보기 (둘 다 비용 0원)

AI 답을 조절하는 손잡이: 파라미터



`--model`

어떤 AI 모델을 쓸지

기본 claude-opus-5-5



`--effort`

답하기 전에 얼마나 깊이 생각할지

기본 low (요약엔 충분)



`--max-tokens`

답의 최대 길이 (생각한 양 포함)

기본 16000



`--temperature`

답의 무작위성 (높을수록 다양)

현재 API는 받지 않음

알게 된 점: 과제 예시의 temperature는 최신 API에서 지원이 끝났습니다. 그래서 기본은 보내지 않고, 같은 역할을 effort로 대신합니다.

안전 모드: 비밀은 가리고, 양은 줄이고

끄면 (OFF)

```
+API_KEY = "sk-ant-api03-abcd..."
+ADMIN_EMAIL = "admin@example.com"
+SUPPORT_PHONE = "010-1234-5678"
```

켜면 (ON)

```
+API_KEY = "[MASKED_ANTHROPIC_KEY]"
+ADMIN_EMAIL = "[MASKED_EMAIL]"
+SUPPORT_PHONE = "[MASKED_PHONE]"
```

10개

보내는 최대 파일 수

200줄

보내는 최대 diff 줄 수

7종+

가리는 패턴 (키·토큰·이메일·전화 등)

기준은 --max-files, --max-diff-lines, --mask-pattern 옵션으로 바꿀 수 있습니다. (화면 속 키는 시연용 가짜 값)

AI 답이 형식에 안 맞으면?



정리

Summary, * 불릿 같은 표기 차이를 표준 형식으로 맞춤



검사

제목 72자 이하? 4개 섹션? 섹션마다 불릿?



1회 재요청

틀린 점을 알려 주고 한 번만 다시 요청



후처리 보정

그래도 틀리면 제목 자르기, 빈 섹션에 '직접 작성' 표시

최대 2회

1회 실행당 AI 호출 횟수 상한. 비용을 예측할 수 있고, 호출 횟수는 매번 화면에 표시됩니다.

문제가 생기면 원인을 알려 줍니다

상황	화면에 나오는 안내
바뀐 내용이 없음	`변경 사항이 없습니다. 코드 설명을 생성하지 않고 종료합니다.`
API Key 없음	`AI_API_KEY 환경변수가 설정되지 않았습니다. + 설정 예시`
잘못된 키 (401)	`인증 실패(401): API Key가 올바른지 확인하세요.`
요청 한도 초과 (429)	`요청 한도 초과(429): 잠시 후 다시 실행하세요.`
인터넷 연결 끊김	`네트워크 오류: API 서버에 연결할 수 없습니다.`
엉뚱한 폴더에서 실행	`프로젝트 루트 디렉터리에서 실행하세요. (저장소 루트: …)`

이전 버전은 실제 API 오류에서 프로그램이 멈추며 알아보기 힘든 오류(traceback)를 냈습니다. 이번에 모두 원인 메시지로 바꿨습니다.

3

결과는 어땠나

실제 실행 결과, 숫자로 본 결과, 보너스, 배운 점

실제 AI로 이번 변경을 설명해 봤습니다

```
[INFO] Git status 수집 완료: 30개 파일 변경 감지
[INFO] safe-mode ON | 파일 10/30개 · diff 200/2501줄 전송
[INFO] AI API 요청 중... (model=claude-opus-5-5)
[DONE] 코드 설명 생성 완료

--- Description Title --- (43자)
Add AI code description helper CLI for b3-2

## Summary
- Git 변경과 수정 의도를 Claude API에 보내
  4개 섹션 설명 초안을 출력하는 CLI를 추가
- 30개 중 10개 파일만 전송되어 일부는 확인 필요
## What Changed  ## Why ...  ## How to Verify

[OK] 제목 길이/섹션 헤더/섹션별 불릿 검증 통과
[INFO] AI API 호출 횟수: 1회 (최대 2회)
```



1번 만에 통과

AI 호출 1회로 형식 검증 통과



팀 규칙도 지킴

영어 명령형 제목 'Add …' 사용



모르는 건 모른다고

못 본 파일은 '확인 필요'로 표시

숫자로 보는 결과

27개

자동 테스트 모두 통과

11개

실행 결과 증빙 파일

1회

실제 생성에 든 AI 호출

3 / 3

보너스 과제 완료

테스트가 확인하는 것

- Git 수집: 수정·스테이징·새 파일 모두 수집, 루트가 아니면 안내
- 안전 모드: 키·이메일·전화 가리기, 10개 파일/200줄 제한
- 형식: 검사 → 재요청 → 보정, 오류 메시지 변환 (실제 API는 부르지 않음)

보너스 과제 3개

팀 컨벤션

저장소의 커밋 92개를 분석해 팀 작성 규칙 파일을 만들었습니다.

```
전  feat: 세션 검증 보완
후  Add 세션 검증 보완
```

안전 모드 고도화

가릴 패턴을 늘리고, 파일 수·줄 수 기준을 옵션으로 바꿀 수 있게 했습니다.

```
OFF 14개 파일·비밀 노출
ON  10개 파일·3건 가림
```

실제 저장소 적용

이번 작업에 도구를 써서 AI 초안을 만들고, 사람이 다듬어 최종 설명을 완성했습니다.

```
초안 Add … CLI
최종 Improve …
```

증빙: 제출/results/06 (안전 모드 비교) · 07 (컨벤션 전후 비교) · 제출/보너스1_실제저장소_적용.md

AI 초안은 출발점, 마무리는 사람

제목	'새 CLI 추가'로 썼지만 실제로는 기존 도구 보완 → Improve로 수정
What Changed	문서 파일 위주 → 실제 로직이 바뀐 모듈로 교체
Why It Was Needed	수정 의도를 되풀이 → 이전 구현의 구체적인 문제를 근거로
How to Verify	테스트 개수 26개 → 실제 값 27개로 정정



왜 고쳐야 했나

안전 모드가 경로 순서대로 앞 10개 파일만 보내서, 핵심 코드가 AI에 전달되지 않았습니다. AI는 본 만큼만 씁니다.

그래서 출력 마지막에 항상 "생성된 설명은 초안입니다. 검토한 뒤 적용하세요."라고 표시합니다.

이 과제로 설명할 수 있게 된 것



API 연동 흐름

질문 만들기 → 보내기 → 답 해석 → 오류 대응



파라미터의 역할

model·effort·max_tokens가 품질과 비용을 바꿈



Git을 입력으로

status·diff 결과를 그대로 질문지에 연결



프롬프트 설계

의도 + 근거 + 형식을 함께 주면 답이 정확해짐



검증과 다듬기

형식 검사·재요청·보정으로 실무 품질 확보

핵심 한 줄: AI에게 무엇을 보여 주느냐가 결과를 결정한다